EXPERIMENT-4: ES6 features and weather graph

Explore ES6 features like arrow functions, callbacks, promises, async/await. Implement an application for reading weather information from openweathermap.org and display the information in the form of a graph on the web page.

Objective : Use modern JavaScript for asynchronous API requests and visualization.

Tools : ES6, Fetch API, async/await, charting, OpenWeatherMap

Outcome :A weather dashboard that fetches data and shows it as a graph.

Answer:

Fetch weather data, transform it into chart labels and values, and use a chart library to visualize the result.

File Name: index.html

```
<!DOCTYPE html>
<html lang="en">
 <head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Week 4 - Weather Dashboard</title>
  <link
   href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
   rel="stylesheet"
  />
  <link rel="stylesheet" href="styles.css" />
 </head>
 <body>
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark sticky-top">
   <div class="container">
    <a class="navbar-brand fw-bold" href="#">WeatherHub</a>
    <button
     class="navbar-toggler"
     type="button"
     data-bs-toggle="collapse"
     data-bs-target="#navbarNav"
```

```
        aria-controls="navbarNav"
        aria-expanded="false"
        aria-label="Toggle navigation"
      >
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav ms-auto">
          <li class="nav-item"><a class="nav-link active" href="#">Home</a></li>
          <li class="nav-item"><a class="nav-link" href="#dashboard">Dashboard</a></li>
        </ul>
      </div>
    </div>
  </nav>

  <main class="container py-4 py-lg-5">
    <section class="hero-card p-4 p-lg-5 rounded-4 text-white mb-4">
      <div class="row align-items-center g-4">
        <div class="col-lg-8">
          <h1 class="display-6 fw-bold mb-3">Weather insights with modern JavaScript</h1>
          <p class="mb-0">
            Fetch live weather data, process it with promises and async/await, and display it on a
chart.
          </p>
        </div>
      </div>
    </section>

    <section id="dashboard" class="card shadow-sm border-0 p-4">
      <div class="row g-4 align-items-end">
        <div class="col-md-8">
          <label for="cityInput" class="form-label fw-semibold">Enter city name</label>
          <input id="cityInput" class="form-control" placeholder="Example: London" />
        </div>
```

```
        <div class="col-md-4">
          <button id="fetchBtn" class="btn btn-primary w-100">Get Weather</button>
        </div>
      </div>

      <div id="statusMessage" class="mt-3 text-muted"></div>

      <div class="row g-4 mt-2">
        <div class="col-lg-5">
          <div class="weather-card p-4 rounded-4">
            <h2 id="cityName" class="h4 fw-bold">City</h2>
            <p id="temperature" class="display-5 fw-bold mb-2">-- °C</p>
            <p id="description" class="text-capitalize">Weather details will appear here</p>
            <ul class="list-unstyled mt-3 mb-0">
              <li>Humidity: <span id="humidity">--</span>%</li>
              <li>Wind: <span id="wind">--</span> km/h</li>
              <li>Pressure: <span id="pressure">--</span> hPa</li>
            </ul>
          </div>
        </div>
        <div class="col-lg-7">
          <canvas id="weatherChart" height="220"></canvas>
        </div>
      </div>
    </section>
  </main>

  <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.3/dist/chart.umd.min.js"></script>
  <script src="script.js"></script>
 </body>
</html>
```

File Name: styles.css

Code:

```
body {
  background: linear-gradient(135deg, #f8fafc 0%, #eef2ff 100%);
  min-height: 100vh;
}

.hero-card {
  background: linear-gradient(120deg, #2563eb, #4f46e5);
}

.weather-card {
  background: #ffffff;
  border: 1px solid #e5e7eb;
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.06);
}

canvas {
  background: #fff;
  border-radius: 16px;
  border: 1px solid #e5e7eb;
  padding: 12px;
}
```

File Name: script.js

```
const geocodeUrl = 'https://geocoding-api.open-meteo.com/v1/search';
const forecastUrl = 'https://api.open-meteo.com/v1/forecast';

const cityInput = document.getElementById('cityInput');
const fetchBtn = document.getElementById('fetchBtn');
const statusMessage = document.getElementById('statusMessage');
```

```
const cityName = document.getElementById('cityName');
const temperature = document.getElementById('temperature');
const description = document.getElementById('description');
const humidity = document.getElementById('humidity');
const wind = document.getElementById('wind');
const pressure = document.getElementById('pressure');

let weatherChart = null;

const showStatus = (message, type = 'info') => {
  statusMessage.textContent = message;
  statusMessage.className = `mt-3 ${type === 'error' ? 'text-danger' : 'text-muted'}`;
};

const getWeatherDescription = (code) => {
  const descriptions = {
    0: 'Clear sky',
    1: 'Mainly clear',
    2: 'Partly cloudy',
    3: 'Cloudy',
    45: 'Fog',
    48: 'Rime fog',
    51: 'Light drizzle',
    53: 'Drizzle',
    55: 'Heavy drizzle',
    61: 'Light rain',
    63: 'Rain',
    65: 'Heavy rain',
    71: 'Light snow',
    73: 'Snow',
    75: 'Heavy snow',
    95: 'Thunderstorm',
  };
```

```
  return descriptions[code] || 'Weather conditions available';
};

const buildChart = (labels, values) => {
  const ctx = document.getElementById('weatherChart');

  if (weatherChart) {
    weatherChart.destroy();
  }

  weatherChart = new Chart(ctx, {
    type: 'line',
    data: {
      labels,
      datasets: [
        {
          label: 'Temperature (°C)',
          data: values,
          borderColor: '#2563eb',
          backgroundColor: 'rgba(37, 99, 235, 0.2)',
          tension: 0.4,
          fill: true,
          pointRadius: 4,
        },
      ],
    },
    options: {
      responsive: true,
      plugins: {
        legend: { display: true },
      },
      scales: {
```

```
      y: { beginAtZero: false },
    },
  },
 });
};

const fetchWeather = async (city) => {
 showStatus('Fetching weather data...');

 try {
  const geoResponse = await
fetch(`${geocodeUrl}?name=${encodeURIComponent(city)}&count=1&language=en&format
=json`);
  const geoData = await geoResponse.json();

  if (!geoData.results || geoData.results.length === 0) {
   throw new Error('We could not find that city. Please try another city name.');
  }

  const { name, country, latitude, longitude } = geoData.results[0];
  const forecastResponse = await fetch(
   `${forecastUrl}?latitude=${latitude}&longitude=${longitude}&current=temperature_2m,r
elative_humidity_2m,wind_speed_10m,pressure_msl,weather_code&hourly=temperature_
2m&forecast_days=1&timezone=auto`
  );
  const forecastData = await forecastResponse.json();

  const current = forecastData.current;
  const hourlyTimes = forecastData.hourly.time.slice(0, 6);
  const hourlyTemps = forecastData.hourly.temperature_2m.slice(0, 6);
  const labels = hourlyTimes.map((time) => time.split('T')[1].slice(0, 5));

  cityName.textContent = `${name}, ${country}`;
  temperature.textContent = `${Math.round(current.temperature_2m)} °C`;
```

```
    description.textContent = getWeatherDescription(current.weather_code);
    humidity.textContent = current.relative_humidity_2m;
    wind.textContent = current.wind_speed_10m;
    pressure.textContent = current.pressure_msl;

    buildChart(labels, hourlyTemps);
    showStatus('Weather data loaded successfully.');
  } catch (error) {
    showStatus(error.message, 'error');
    cityName.textContent = 'City';
    temperature.textContent = '-- °C';
    description.textContent = 'Weather details will appear here';
    humidity.textContent = '--';
    wind.textContent = '--';
    pressure.textContent = '--';
  }
};

fetchBtn.addEventListener('click', () => {
  const city = cityInput.value.trim();
  if (!city) {
    showStatus('Please enter a city name.', 'error');
    return;
  }
  fetchWeather(city);
});

cityInput.addEventListener('keydown', (event) => {
  if (event.key === 'Enter') {
    event.preventDefault();
    fetchBtn.click();
  }
});
```

```
window.addEventListener('load', () => {
  fetchWeather('Delhi');
});
```